

AI-Based Network Intrusion Detection and Threat Intelligence Dashboard

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

SRIVALLI 2520090040

PAVANI 2520090041

KAVYA SREE 2520090143

Under the Esteemed Guidance of

Dr. Mr. Khaja Shareef

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

Declaration

The Project Report entitled “**AI-Based Network Intrusion Detection and Threat Intelligence Dashboard**” is a record of bonafide work of **SIVALLI-2520090040, PAVANI-2520090041, KAVYASREE-2520090143** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 12, Team 10, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

SRIVALLI – 2520090040

PAVANI-2520090041

KAVYASREE– 2520090143

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

CERTIFICATE

This is to certify that the project report entitled “**AI-Based Network Intrusion Detection and Threat Intelligence Dashboard**” is a bonafide work done and submitted by **SRSIVALLI-2520090040, PAVANI-2520090041, KAVYASREE-2520090143** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 12, Team 10, at KLH University during **2026–2027, Trimester 4**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr** , and my guide **Mr. Khaja Shareef**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

SRIVALLI-2520090040

PAVANI-2520090041

KAVYASREE-2520090143

Abstract

The **AI-Based Network Intrusion Detection and Threat Intelligence Dashboard** is a web-based security application designed to help organizations monitor network traffic and identify potential cyber threats. Traditional security systems may have difficulty detecting evolving and suspicious network activities, making continuous monitoring and analysis challenging. These issues can lead to delayed threat detection and increased security risks.

The proposed system provides a user-friendly dashboard through which users can monitor network activity, detect intrusions, and view classified attack information. The system uses **Artificial Intelligence and Machine Learning techniques**, including **Random Forest and Deep Neural Network (DNN)** models, to analyze network traffic and identify malicious patterns. It also provides threat alerts and security insights to support faster threat monitoring and analysis.

The application uses **HTML, CSS, and JavaScript** for the frontend, **Python and Flask** for the backend, and **MySQL** for storing network and threat information. The system includes modules for data preprocessing, feature extraction, AI-based intrusion detection, attack classification, threat monitoring, and dashboard visualization. The system is developed and tested using **VS Code**, with machine learning libraries such as **Scikit-learn and TensorFlow**. The main goal of the system is to automate network threat detection, improve security monitoring, and provide a centralized platform for analyzing and managing intrusion-related information.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	9

5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	10
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	12
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	13
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	15
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	17
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	18
S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	19
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	20
13	References	– Documentation, tools, and resources used	21
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

With the rapid growth of internet-connected systems, networks are continuously exposed to different types of cyber threats and malicious activities. Traditional security methods mainly depend on predefined rules and signatures, which may not effectively identify new or suspicious attack patterns.

The AI-Based Network Intrusion Detection and Threat Intelligence Dashboard is developed to address these challenges by using Artificial Intelligence and Machine Learning techniques. The system analyzes network traffic, detects suspicious activities, classifies possible attacks, and presents the results through a centralized dashboard.

The system uses Random Forest and Deep Neural Network (DNN) models for intrusion detection and classification. It provides security information such as attack types, threat status, and detection results, helping users monitor network security more efficiently.

1.2 Problem Statement

Network environments generate a large amount of traffic, making manual monitoring difficult and time-consuming. Traditional intrusion detection systems may mainly focus on known attack signatures and can have limitations when dealing with new or evolving attack patterns.

There is a need for an automated system that can analyze network traffic, detect suspicious activities, classify different types of attacks, and present threat information in an easy-to-understand manner. The proposed system addresses this gap using AI-based detection and a centralized security dashboard.

1.3 Scope of the Project

The system covers the complete process of network intrusion detection, starting from the collection and preprocessing of network traffic data to the identification and classification of potential threats. Relevant features are extracted from the network traffic and used to train and test Random Forest and Deep Neural Network (DNN) models. The system detects normal and malicious activities and classifies the detected attacks into different categories. The results are presented through a web-based dashboard, where users can monitor threats, view security alerts, and analyze attack information. The system also stores prediction and threat-related data in a database for further monitoring and analysis.

1.4 Objectives

- To develop an AI-based system for detecting network intrusions.

- ❑ To analyze and classify network traffic using **Random Forest and DNN models**.
- ❑ To identify suspicious and malicious network activities.
- ❑ To provide a centralized dashboard for monitoring detected threats.
- ❑ To store and manage intrusion-related information efficiently.
- ❑ To provide visual threat information for easier security analysis.

1.5 Importance of the Application

The application helps users monitor network activities and identify potential security threats in a centralized environment. By using AI-based models, the system can automate the analysis of network traffic and reduce the need for continuous manual monitoring.

The dashboard provides clear information about detected attacks, threat status, and classification results, allowing users to understand network security events more easily. This can support faster identification and analysis of suspicious activities.

1.6 Target Users / Audience

The system is mainly intended for:

Network Administrators – to monitor network traffic and detected threats.

Cybersecurity Professionals – to analyze attack patterns and security events.

Security Teams – to monitor alerts and intrusion detection results.

Organizations – to improve centralized network security monitoring.

Students/Researchers – for learning and experimenting with AI-based intrusion detection systems.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application has modest hardware requirements, since both the Node.js backend and the MySQL database are lightweight relative to the small dataset a single campus generates. A development or demonstration machine with a dual-core processor, 4 GB of RAM, and roughly 500 MB of free disk space (for Node modules, MySQL data files, and project source) is sufficient to run the entire stack — frontend, backend, and database — on one computer.

2.2 Software Requirements

- Node.js 18 or later, with npm
- MySQL 8.0 or later
- A modern web browser (Chrome, Edge, or Firefox)
- Git, for version control
- MySQL Workbench and Visual Studio Code, for development (optional but recommended)

2.3 Development Tools and Frameworks

React.js (bootstrapped with Vite) was used for the frontend; Node.js with the Express.js framework for the backend REST API; MySQL as the relational database, accessed through the mysql2 driver; Postman for API testing and documentation; and Git with GitHub for source control. VS Code was the primary development environment, and MySQL Workbench was used for inspecting and verifying the database schema and data directly.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a web-based interface developed using HTML, CSS, and JavaScript. It provides users with a simple dashboard for viewing network activity, detected intrusions, attack classifications, threat alerts, and security statistics. The interface communicates with the Flask backend through HTTP requests and displays the prediction and threat information in an organized manner.

3.2 Backend

The backend is developed using **Python and Flask** and acts as the main processing layer of the application. It handles requests from the frontend, performs data processing, communicates with the machine learning models, and manages database operations. Flask APIs are used to send network traffic data to the detection system and return prediction results to the dashboard..

3.3 Database

MySQL is used to store and manage application data such as users, network traffic, predictions, detected threats, attack types, and timestamps. The database maintains relationships between these records using primary keys and foreign keys. Validation constraints are used to maintain data consistency and integrity.

3.4 Version Control

Git was used throughout development for version control, with the project structured as a single repository containing both the client and server codebases, hosted on GitHub.

3.5 Authentication and Security Libraries

The application provides authentication to restrict access to the security dashboard. User credentials are securely handled using password hashing techniques rather than storing plain-text passwords. Backend validation is used to verify incoming requests and prevent invalid data from being stored in the database. Access control can also be applied to restrict administrative functions.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a layered architecture that separates the user interface, backend processing, AI/ML models, and database. Network traffic or dataset information is first processed and converted into suitable features. The processed data is then passed to the Random Forest and DNN models for intrusion detection and attack classification. The results are stored in the database and displayed

through the web dashboard.



Figure 4.1: High-level three-tier architecture of the Event and Room Occupancy Management System

4.2 Description of Each Layer

The **Presentation Layer** consists of the HTML, CSS, and JavaScript-based web dashboard. It allows users to view detected threats, attack classifications, security alerts, and analysis results. The frontend communicates with the backend through HTTP requests and does not directly access the database.

The **Application Layer** is implemented using Python and Flask. It receives requests from the frontend, validates the input, processes network data, communicates with the machine learning

models, and manages database operations. This layer acts as the connection between the user interface, AI models, and database.

The **AI/ML Layer** contains the Random Forest and Deep Neural Network models. Network traffic data is first preprocessed and relevant features are extracted before being passed to the models. The models identify whether the traffic is normal or malicious and classify detected activities into different attack categories.

The **Data Layer** consists of the MySQL database. It stores user information, network traffic records, prediction results, detected threats, attack types, confidence values, and timestamps. Primary keys and foreign keys are used to maintain relationships and data consistency.

.

4.3 Deployment Architecture

The system begins by receiving network traffic data or a prepared intrusion detection dataset. The data is cleaned and preprocessed to remove inconsistencies and convert it into a suitable format. Relevant network features are then extracted and provided to the trained Random Forest and DNN models.

The models analyze the input and generate a prediction indicating whether the network activity is normal or malicious. If suspicious activity is detected, the system identifies the corresponding attack category and generates threat information. The prediction results are stored in the MySQL database and displayed on the web dashboard through the Flask backend

CHAPTER 5: DESIGN

5.1 Database Design Principles

The schema was normalized to remove redundant data: each fact is stored in exactly one place, and every relationship between entities is expressed through a foreign key rather than by duplicating data across tables. Every table enforces its own invariants directly in the database — not just in application code — using PRIMARY KEY, FOREIGN KEY, UNIQUE, and CHECK constraints, so that invalid data (a negative capacity, an end time before a start time, a duplicate registration) is rejected at the database level even if a bug elsewhere in the application tried to insert it.

5.2 ER Diagram / Database Schema

The database consists of seven related tables, shown in Figure 5.1.

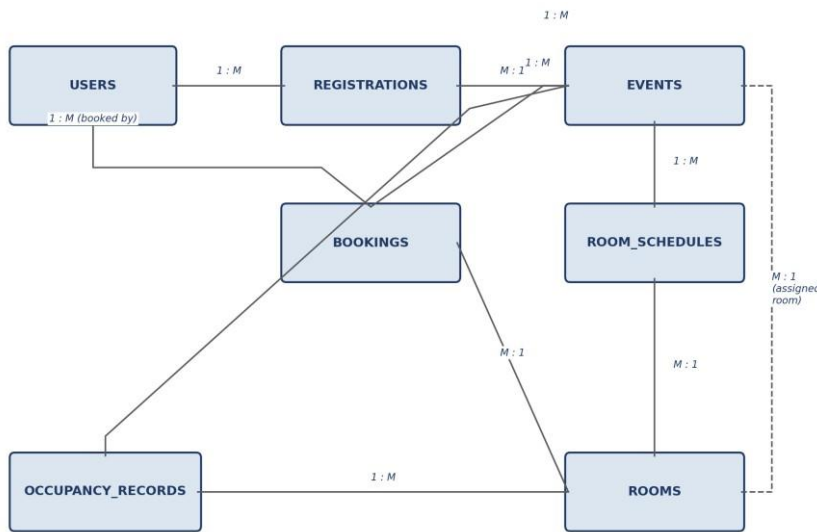


Figure 5.1: Entity-Relationship diagram of the 7-table database schema

The ER diagram and database schema define the structure and relationships of the data used in the AI-Based Network Intrusion Detection and Threat Intelligence Dashboard. The database contains entities such as User, Network Traffic, Prediction, and Threat. Each entity has a unique primary key, while foreign keys are used to establish relationships between related records. Network traffic data is analyzed by the AI models, and the resulting predictions are stored with information such as model name, prediction result, confidence value, and timestamp. Detected threats contain details such as attack type, severity, source IP, destination IP, and detection time. This relational design helps maintain data consistency, reduce redundancy, and support efficient threat monitoring and analysis.

5.3 Design Decisions and Assumptions

Two design decisions were made explicitly, where the original entity list left some ambiguity, and are recorded here rather than left implicit in the code:

- **ROOM_SCHEDULES vs. BOOKINGS:** since both tables could plausibly record “room + time,” ROOM_SCHEDULES was designated the single source of truth that every availability check and every room-assignment reads from, while BOOKINGS is treated purely as an audit trail of who performed the assignment. Both are written together, atomically, inside one database transaction, so they can never drift out of sync.
- **Effective registration capacity:** the EVENTS table has no dedicated “maximum participants” field, so once a room is assigned to an event, that room's capacity is treated as the effective registration limit. Before a room is assigned, registration is left uncapped.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The machine learning module is implemented using Python and Scikit-learn. Network intrusion data is first loaded and preprocessed by handling missing values, encoding categorical features, and selecting relevant network attributes. The processed dataset is divided into training and testing sets. A Random Forest classifier is trained to identify normal and malicious network traffic and classify different attack categories. The trained model is evaluated using suitable performance metrics and saved for prediction.

6.2 Front-end Logic

A Deep Neural Network (DNN) is implemented using TensorFlow/Keras to identify complex patterns in network traffic. The input features are normalized before being provided to the neural network. The model consists of input, hidden, and output layers and is trained using the prepared intrusion detection dataset. After training, the model is tested using unseen data and its prediction results are used for threat detection and attack classification.

6.3 API Integration

The preprocessing module prepares raw network traffic data for the AI models. It handles missing or inconsistent values, converts categorical information into numerical form, and removes unnecessary attributes. Important network features such as protocol, source information, destination information, packet-related values, and traffic characteristics are selected for model training. The processed features are then passed to the Random Forest and DNN models.

6.4 Authentication and Authorization

The backend is developed using **Python and Flask**. Flask provides APIs that connect the frontend dashboard with the machine learning models and MySQL database. When network data is submitted, the backend preprocesses the input and sends it to the trained models for prediction. The prediction result, attack category, confidence value, and related threat information are then returned to the dashboard and stored in the database..

CHAPTER 7: FEATURES

7.1 List of Main Features Student-facing

features:

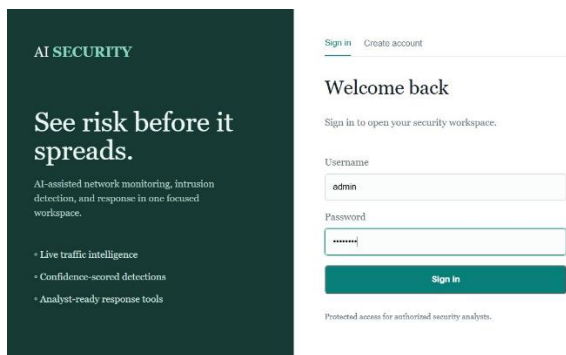
- ☐ Upload and process network intrusion datasets.
- ☐ Preprocess and clean network traffic data automatically.
- ☐ Extract relevant features for intrusion detection.
- ☐ Detect normal and malicious network activities.
- ☐ Use Random Forest and DNN models for prediction.
- ☐ Classify detected activities into different attack categories.
- ☐ Display prediction confidence and threat severity.
- ☐ View detected threats through a centralized dashboard.
- ☐ Store network traffic and prediction results in the database.
- ☐ Display security statistics and attack information.
- ☐ Provide threat alerts for suspicious activities.
- ☐ Monitor source and destination IP information.
- ☐ View detection time and threat status.
- ☐ Support analysis of historical intrusion records.

7.2 How Each Feature Works

AI-BasedIntrusionDetection:

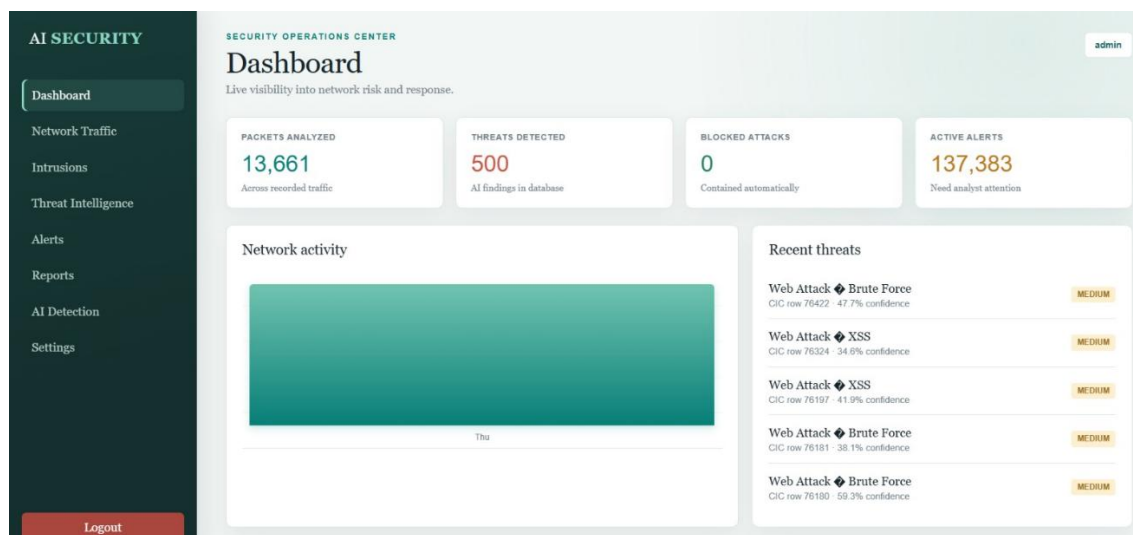
User Authentication

The system provides a secure login page where authorized users enter their username and password to access the security workspace. After successful authentication, the user is redirected to the main dashboard. The **Sign in** and **Create account** options provide controlled access to the application.



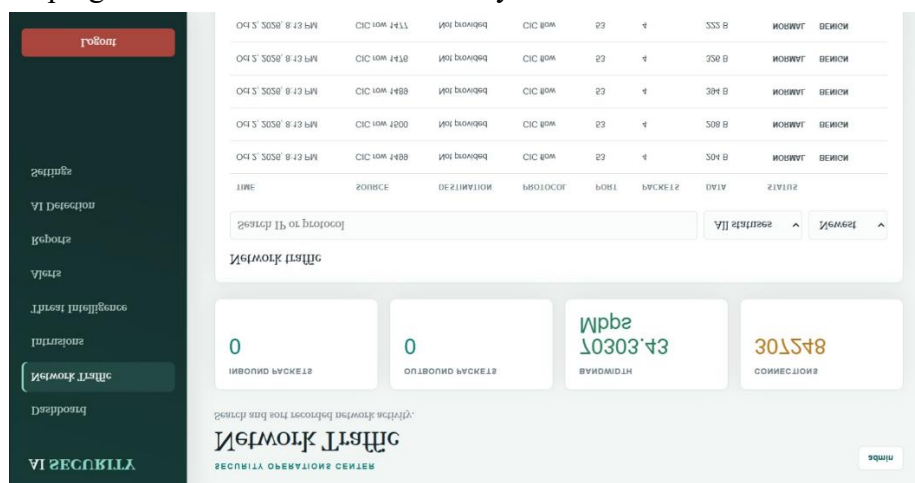
Security Dashboard

The dashboard provides a centralized overview of network security activity. It displays important statistics such as **packets analyzed, threats detected, blocked attacks, and active alerts**. It also shows recent threats with their attack type and confidence level, allowing users to quickly understand the current security status



Network Traffic Monitoring

The Network Traffic module displays recorded network activity in a structured table. Users can search traffic using IP addresses or protocols and filter records by status. The page provides information such as time, source, destination, protocol, port, packets, data, and traffic status, helping users examine network activity.



AI-Based Intrusion Detection

The AI Detection module allows the user to upload a CSV network traffic file for analysis. After clicking Analyze Traffic, the system processes the uploaded records using the trained AI model. It displays the number of records analyzed, attack predictions, prediction results, source and destination information, and the model confidence percentage.

Threat Detection and Classification

After analyzing the network traffic, the AI model identifies whether the traffic is BENIGN or malicious. Suspicious traffic can be classified into attack categories such as Brute Force, XSS, or other supported attack types. The prediction confidence is also displayed so users can understand the model's output.

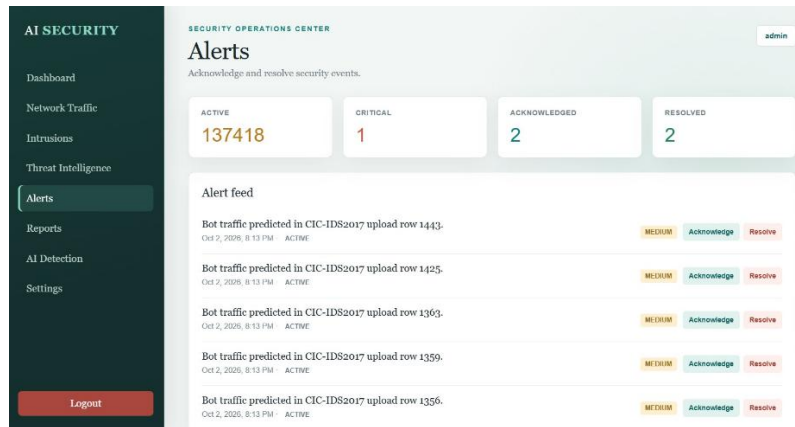
Alerts Management

The Alerts module collects security events generated from the detection process. Each alert shows the detected event, time, current status, and severity. Users can take actions such as

Acknowledge or Resolve an alert, allowing security events to be tracked from detection to resolution.

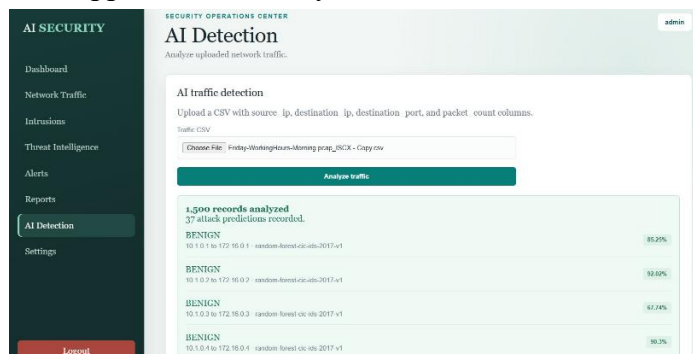
Threat Intelligence

The Threat Intelligence section is used to organize and present information related to detected threats. It helps users examine attack information and understand the security risks identified by the intrusion detection system.



Reports

The **Reports** module is intended to provide summarized security information based on the recorded network traffic, detected intrusions, and alerts. It can be used to review security activity and support further analysis.



Logout and Settings

The **Logout** option allows the authenticated user to safely exit the security workspace. The **Settings** section provides a place for managing application or user-related configuration.

Overall Working Flow

Login → Dashboard → Network Traffic → Upload CSV → AI Detection → Attack Classification → Threat/Alert Generation → Monitoring & Resolution

CHAPTER 8: TESTING

8.1 Postman API Testing

The AI-based intrusion detection system was tested using network traffic data to verify the performance of the trained Random Forest and Deep Neural Network (DNN) models. The testing process checks whether the models can correctly identify normal and malicious network activities and classify detected attacks into their respective categories. The prediction results are evaluated using measures such as accuracy, precision, recall, and F1-score.

8.2 Integration Testing

The complete system was tested end-to-end to verify the interaction between the frontend, Flask backend, AI models, and MySQL database. The testing covers CSV file upload, data preprocessing, model prediction, attack classification, database storage, dashboard display, and alert generation. The complete workflow was also checked to ensure that the prediction results generated by the AI model are correctly displayed on the dashboard and stored in the database.

8.3 Tools Used for Testing

- VS Code — development and manual testing of the complete application.
- Web Browser — testing login, dashboard, network traffic, alerts, and AI detection modules.
- Python — testing data preprocessing and machine learning model predictions.
- Flask — testing backend APIs and communication between modules.
- MySQL — verifying stored traffic, prediction, and threat records.
- CSV Dataset — testing network traffic analysis and intrusion detection.

8.4 Test Cases and Results

A representative sample of the test cases executed, drawn from the project's actual test log, is summarized below.

#	Test Case	Expected Result	Actual Result
1	Login with wrong password	User successfully enters dashboard	Pass
2	Login with invalid credentials	Login should be rejected	Pass
3	Register for the same event twice	Recorded network traffic should be displayed	Pass
4	Search network traffic by IP/protocol	Matching traffic records should be displayed	Pass
5	Upload valid CSV file	CSV file should be accepted for analysis	Pass
6	Analyze uploaded network traffic	network traffic Records should be processed by the AI model	Pass

7	Detect normal network traffic	Traffic should be classified as BENIGN	Pass
#	Test Case	Expected Result	Actual Result
8	Detect malicious network traffic	Suspicious traffic should be identified	Pass
9	Classify detected attack	Attack should be assigned to the appropriate category	Pass
10	Display prediction confidence	Confidence percentage should be displayed	Pass
11	Generate security alert	Detected threat should appear in the Alerts module	Pass
12	Resolve an alert	Alert should be marked as resolved	Pass
13	Display recent threats	Latest detected threats should appear on dashboard	Pass
14	Dashboard statistics	Packets, threats, alerts and other statistics should be displayed	Pass

The testing confirms that the major modules of the system work together as expected. The application successfully supports user authentication, network traffic monitoring, CSV-based AI detection, attack classification, confidence-based predictions, threat alerts, and database storage.

The end-to-end workflow from network traffic upload → AI analysis → prediction → threat detection → alert generation → dashboard display was verified

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration as follows:

- Load the schema and seed data: `mysql -u root -p < database/schema.sql`, then ... `< database/seed.sql`
- Start the backend: `cd server, npm install, npm run dev` (listens on port 5000)
- Start the frontend: `cd client, npm install, npm run dev` (listens on port 5173)
- Open `http://localhost:5173` in a browser and log in with a seeded account

9.2 Environment Configuration

All server configuration is kept in a single `.env` file (never committed to version control), covering the database connection (host, port, user, password, database name), the JWT signing secret and token expiry, the port the server listens on, and the frontend's URL for CORS. A `.env.example` template with placeholder values is committed instead, so the required variables are documented without exposing any real credentials.

9.3 Hosting of Frontend and Backend

The **frontend** of the AI-Based Network Intrusion Detection and Threat Intelligence Dashboard can be hosted on a web hosting platform, allowing users to access the dashboard through a browser. The **backend**, developed using **Python and Flask**, can be deployed on a server to handle API requests, AI model predictions, and communication with the database. The frontend communicates with the backend through REST APIs, while **MySQL** stores network traffic, prediction, and threat information. This deployment structure allows the complete application to operate as a centralized web-based security system.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Dataset Quality:** Network traffic datasets may contain missing values, duplicate records, and inconsistent features, which can affect prediction accuracy.
- **Model Training:** Training Random Forest and Deep Neural Network models requires suitable preprocessing, feature selection, and parameter tuning.
- **Attack Classification:** Some malicious traffic patterns may resemble normal activities, making accurate classification challenging.
- **Frontend–Backend Integration:** Connecting the web dashboard with Flask APIs, machine learning models, and the MySQL database requires proper data handling.
- **Prediction Confidence:** Confidence percentages may vary depending on the quality of the dataset and the trained model.
- **Threat Alert Management:** Displaying detected threats and maintaining correct alert statuses require proper coordination between the detection and database modules.

10.2 Solutions Applied

Data preprocessing techniques were used to clean the dataset, handle missing values, and convert features into a suitable format for model training. Random Forest and DNN models were trained and evaluated to identify normal and malicious network activities. CSV input validation helps ensure that uploaded files contain the required columns. Flask APIs connect the frontend with the machine learning models and database, while structured database records support threat monitoring and prediction history. Dashboard testing was performed to verify the display of prediction results and security alerts..

For the MySQL startup failure, the stale process was terminated, the leftover socket and PID files were removed, and the service was restarted cleanly — now documented as a known troubleshooting step in the project's README.

10.3 Current Limitations or Known Bugs

- ☐ The system primarily analyzes uploaded CSV files rather than continuously monitoring live network traffic.
- ☐ Prediction accuracy depends on the quality, variety, and representativeness of the training dataset.
- ☐ Unknown or newly developed attack patterns may not be classified correctly.
- ☐ Automated blocking of malicious traffic is not implemented unless connected to suitable network security controls.
- ☐ Threat intelligence depends on the information available to the system and does not necessarily include live external threat feeds.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- ☐ **Real-time network monitoring:** Integrate live network traffic capture for continuous intrusion detection.
- ☐ **Advanced threat alerts:** Add email or SMS notifications for critical security events.
- ☐ **Automated threat response:** Integrate with firewalls or other security tools to support blocking confirmed malicious traffic.
- ☐ **Enhanced threat visualization:** Provide detailed charts, attack trends, and historical threat reports.
- ☐ **User access management:** Introduce role-based access for administrators and security analysts.

11.2 Possible Integrations or Optimizations

- ☐ Integrate external threat intelligence feeds to obtain updated information about malicious IP addresses and attack patterns.
- ☐ Optimize Random Forest and DNN models to improve prediction speed and detection performance.
- ☐ Deploy the frontend and Flask backend on cloud hosting platforms for remote access.
- ☐ Add automated testing to verify model predictions, API functionality, and database operations after updates.
- ☐ Implement secure API communication, stronger authentication, and database backup mechanisms to improve system reliability.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project presents an AI-Based Network Intrusion Detection and Threat Intelligence Dashboard designed to identify and monitor suspicious network activities through a centralized web application. The system combines network traffic analysis with Random Forest and Deep Neural Network (DNN) models to detect normal and malicious activities. A web-based dashboard provides users with information about detected threats, network traffic, alerts, prediction confidence, and attack categories. The system was developed and tested by integrating the frontend, backend, AI models, and database into a single security monitoring platform.

12.2 What Was Achieved

The completed system successfully provides AI-based network traffic analysis, intrusion detection, attack classification, and threat monitoring. Users can upload network traffic data in CSV format, analyze the records using the trained AI model, and view prediction results and confidence values. The dashboard displays network activity, detected threats, and active alerts, while the alert module allows security events to be acknowledged and resolved. The system also stores relevant prediction and threat information for further analysis.

12.3 Skills Learned

Working on this project provided practical experience in machine learning, network intrusion detection, data preprocessing, and web application development. The project involved training and testing Random Forest and DNN models, working with network traffic datasets, developing Flask-based backend APIs, designing a security dashboard, and connecting the application with a database. It also provided experience in testing the complete workflow from data upload and AI prediction to threat detection, alert generation, and dashboard visualization.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev>
- Vite — Official Documentation, <https://vitejs.dev>
- Node.js — Official Documentation, <https://nodejs.org/en/docs>
- Express.js — Official Documentation, <https://expressjs.com>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- mysql2 (Node.js MySQL driver) — npm documentation, <https://www.npmjs.com/package/mysql2>
- jsonwebtoken — npm documentation, <https://www.npmjs.com/package/jsonwebtoken>
- bcrypt — npm documentation, <https://www.npmjs.com/package/bcrypt>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs — JavaScript and Web APIs reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

The core room-conflict overlap query, used consistently across the system:

```
import pandas as pd
import joblib
# Load trained intrusion detection model
model = joblib.load("intrusion_model.pkl")
# Load network traffic data
data = pd.read_csv("network_traffic.csv")
# Select required features
features = data[FEATURE_COLUMNS]
# Generate predictions
predictions = model.predict(features)
# Store prediction results
data["Prediction"] = predictions
print(data[["Prediction"]])
```

Appendix B: Installation and Setup Instructions

- `mysql -u root -p < database/schema.sql`
- `mysql -u root -p < database/seed.sql`
- `cd server && npm install && npm run dev` (backend, port 5000)
- `cd client && npm install && npm run dev` (frontend, port 5173)
- Open `http://localhost:5173` in a browser

Appendix C: Default Login Credentials

All seeded accounts use the password “password123”.

Role	Email
Admin	kavya@gmail.com
user	user@gmail.com

Appendix D: API Reference Summary

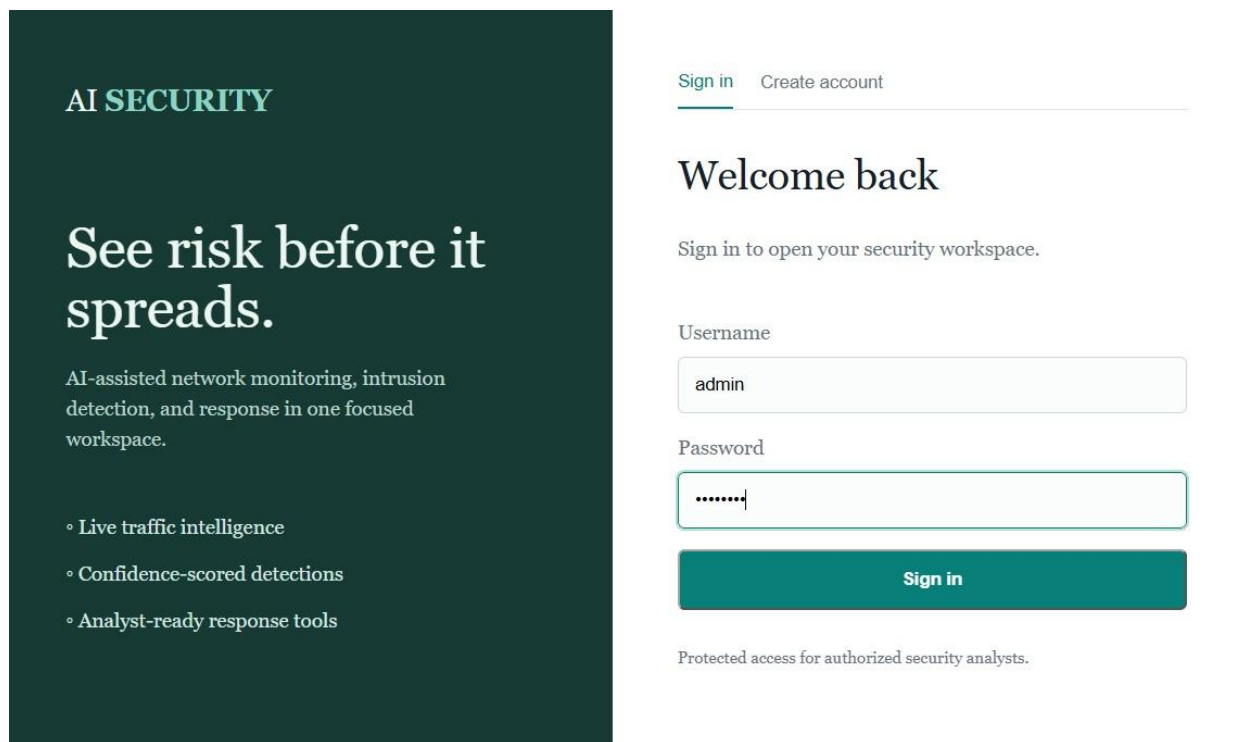
22 endpoints across six resources; full request/response examples are in the accompanying Postman collection.

Resource	Endpoints
Authentication	POST /login, POST /register
Dashboard	GET /dashboard, GET /statistics
Network Traffic	GET /traffic, GET /traffic/search
AI Detection	POST /analyze, POST /predict
Threats	GET /threats, GET /threats/:id

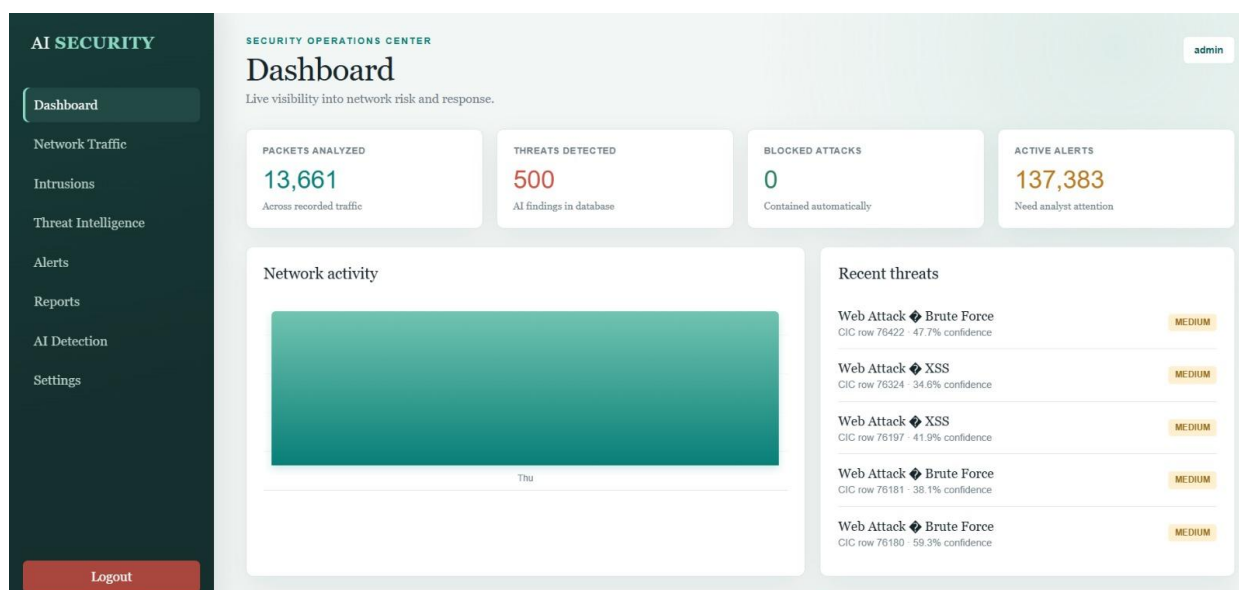
Alerts	GET /alerts, PUT /alerts/:id/acknowledge, PUT /alerts/:id/resolve
Reports	GET /reports

Appendix E: Screenshots

The following pages are left for the student to complete with actual screenshots taken from the running application before submission. Suggested screenshots, in order: 1. Login Page



2. Dashboard



3. Network Traffic

AI SECURITY

Dashboard

Network Traffic

Intrusions

Threat Intelligence

Alerts

Reports

AI Detection

Settings

Logout

SECURITY OPERATIONS CENTER

admin

Network Traffic

Search and sort recorded network activity.

INBOUND PACKETS

0

OUTBOUND PACKETS

0

BANDWIDTH

70303.43

Mbps

CONNECTIONS

307248

Network traffic

Search IP or protocol

All statuses

Newest

TIME	SOURCE	DESTINATION	PROTOCOL	PORT	PACKETS	DATA	STATUS	
Oct 2, 2026, 8:13 PM	CIC row 1499	Not provided	CIC flow	53	4	204 B	NORMAL	BENIGN
Oct 2, 2026, 8:13 PM	CIC row 1500	Not provided	CIC flow	53	4	208 B	NORMAL	BENIGN
Oct 2, 2026, 8:13 PM	CIC row 1489	Not provided	CIC flow	53	4	394 B	NORMAL	BENIGN
Oct 2, 2026, 8:13 PM	CIC row 1476	Not provided	CIC flow	53	4	326 B	NORMAL	BENIGN
Oct 2, 2026, 8:13 PM	CIC row 1477	Not provided	CIC flow	53	4	222 B	NORMAL	BENIGN

4. Alerts

AI SECURITY

Dashboard

Network Traffic

Intrusions

Threat Intelligence

Alerts

Reports

AI Detection

Settings

Logout

SECURITY OPERATIONS CENTER

admin

Alerts

Acknowledge and resolve security events.

ACTIVE

137418

CRITICAL

1

ACKNOWLEDGED

2

RESOLVED

2

Alert feed

Bot traffic predicted in CIC-IDS2017 upload row 1443.

Oct 2, 2026, 8:13 PM · ACTIVE

MEDIUM

Acknowledge

Resolve

Bot traffic predicted in CIC-IDS2017 upload row 1425.

Oct 2, 2026, 8:13 PM · ACTIVE

MEDIUM

Acknowledge

Resolve

Bot traffic predicted in CIC-IDS2017 upload row 1363.

Oct 2, 2026, 8:13 PM · ACTIVE

MEDIUM

Acknowledge

Resolve

Bot traffic predicted in CIC-IDS2017 upload row 1359.

Oct 2, 2026, 8:13 PM · ACTIVE

MEDIUM

Acknowledge

Resolve

Bot traffic predicted in CIC-IDS2017 upload row 1356.

Oct 2, 2026, 8:13 PM · ACTIVE

MEDIUM

Acknowledge

Resolve

5. AI Detection

AI SECURITY

Dashboard

Network Traffic

Intrusions

Threat Intelligence

Alerts

Reports

AI Detection

Settings

Logout

SECURITY OPERATIONS CENTER

admin

AI Detection

Analyze uploaded network traffic.

AI traffic detection

Upload a CSV with source_ip, destination_ip, destination_port, and packet_count columns.

Traffic CSV

Choose File

Friday-WorkingHours-Morning.pcap_ISCX - Copy.csv

Analyze traffic

1,500 records analyzed

37 attack predictions recorded.

BENIGN	10.1.0.1 to 172.16.0.1 · random-forest-cic-ids-2017-v1	85.25%
BENIGN	10.1.0.2 to 172.16.0.2 · random-forest-cic-ids-2017-v1	92.02%
BENIGN	10.1.0.3 to 172.16.0.3 · random-forest-cic-ids-2017-v1	67.74%
BENIGN	10.1.0.4 to 172.16.0.4 · random-forest-cic-ids-2017-v1	90.3%